TRAVELGENIE: AI-POWERED TRAVEL PLANNING ECOSYSTEM

DATA/MSML 650: Cloud computing

Presenters:

- **Arunbh Yashaswi**
- **Abhishek Rithik Origanti**
- **Ajaykumar Balakannan**
- **Matheshwara Annamalai Senthilkumar**

The Challenge of Modern Travel Planning

- Planning a trip requires coordinating multiple services
- Flights, hotels, events, weather, currency, traffic data
- Each service has different APIs, formats, and requirements
- Manual coordination is time-consuming and error-prone
- Users need intelligent, context-aware recommendations

TravelGenie - One Intelligent System

- AI-Powered Orchestration: Claude AI coordinates 7 specialized services automatically
- Context-Aware: Remembers conversation history and builds on previous interactions
- Slack Integration: Accessible via Slack bot with real-time responses and team collaboration
- Real-Time Data: Live flight prices, weather forecasts, traffic conditions
- Budget Management: Automatic currency conversion and budget tracking

LITERATURE SURVEY & RELATED WORK (1/2)

Microservices Theory vs. Implementation

Source: Dragoni et al. (2017) – *Microservices: Yesterday, Today, and Tomorrow*

Dragoni et al. (2017) describe microservices as a natural evolution of Service-Oriented Computing, defining them simply as independent processes that communicate via messages to solve the "dependency hell" found in clunky monolithic apps.They argue that success depends on people as much as code: teams should be organized to "build and run" their own specific features, supported by heavy automation. The authors are honest about the downsides, noting that relying on network messages instead of internal memory calls creates lag and makes testing much harder. Finally, they warn that as these systems spread out, we need better security and stricter ways to ensure services actually understand each other's data.

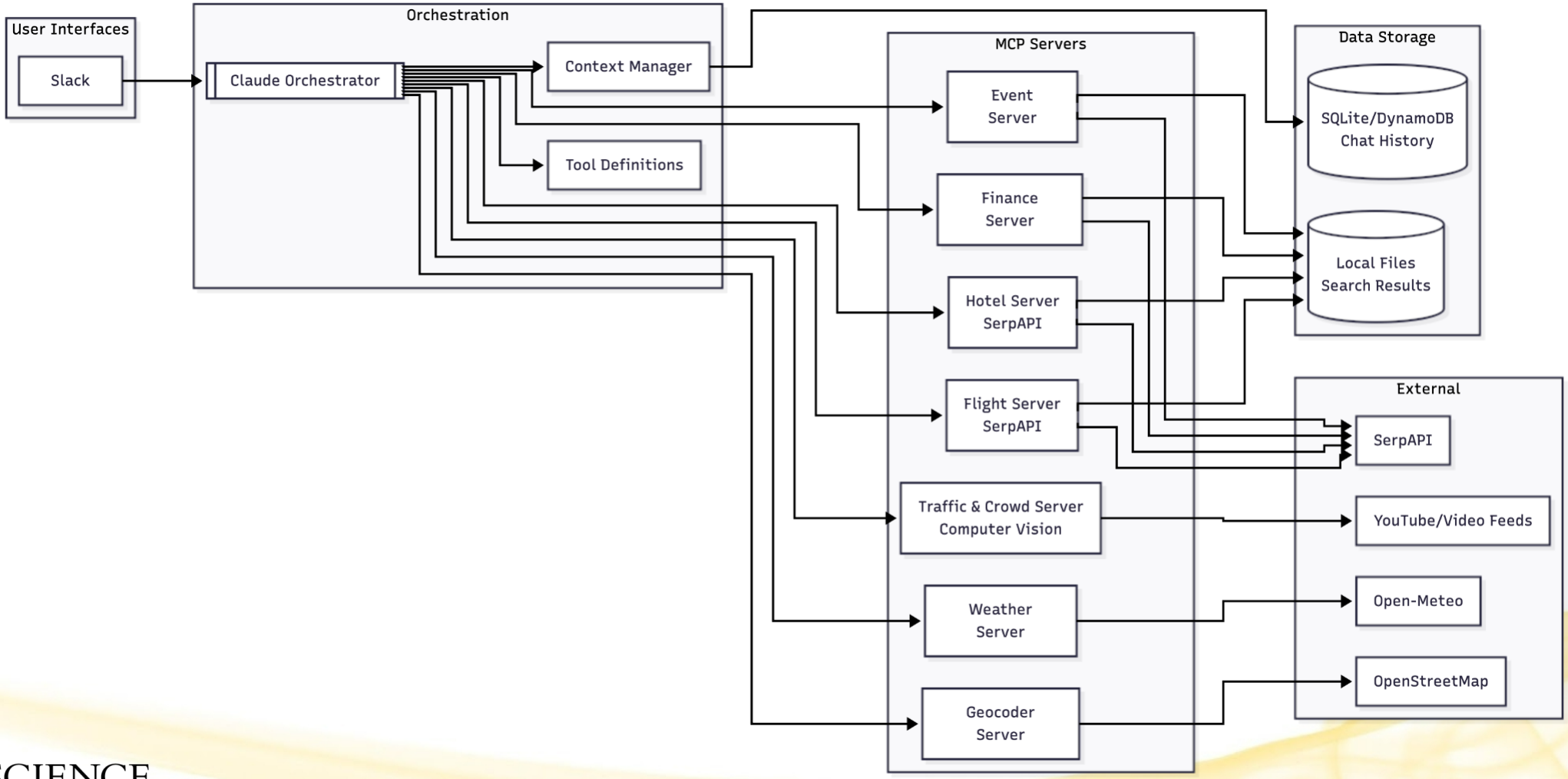
LITERATURE SURVEY & RELATED WORK (2/2)

Serverless Theory vs. Implementation

Source: Jonas et al. (2019) – *Cloud Programming Simplified: A Berkeley View on Serverless Computing*

Jonas et al. (2019) frame serverless computing as a major architectural leap, comparing it to the historic transition from assembly code to high-level programming languages. They define the core innovation as the complete decoupling of computation from storage, which allows developers to focus purely on business logic without ever managing the underlying infrastructure or paying for idle resources.They admit, however, that this abstraction isn't free: the stateless nature of these functions makes handling data-intensive tasks and low-latency communication significantly harder than in traditional setups. Despite these current "growing pains," the authors predict that serverless will eventually become the default model for cloud computing, relegating traditional server management to a niche role for specialized performance needs.

System Architecture Overview



Slackbot to Access TravelGenie

How:

- Direct message @travelgenie or mention in channels
- Natural language queries (e.g., "Plan a trip to Paris next month")

Why Slack:

- Team collaboration in shared workspaces
- Context-aware conversations with history
- Real-time responses via WebSocket
- Enterprise-grade security and scalability
- Mobile and desktop access

Intelligent Orchestration with Claude AI (1/2)

How It Works:

- User submits trip request (origin, dates, budget, interests)
- Orchestrator loads tool definitions (MCP protocol)
- Claude AI plans execution sequence based on dependencies
- Executes tools in parallel when possible
- Synthesizes results into comprehensive itinerary

Intelligent Orchestration with Claude AI (2/2)

Key Features:

- Dependency Resolution: Knows geocoding must happen before weather
- Parallel Execution: Runs independent searches simultaneously
- Context Management: Maintains conversation history
- Budget Tracking: Monitors costs across all services
- Intelligent Filtering: Matches results to user preferences

Specialized Domain Servers

Server Name	Purpose	Data Source
Flight Server	Find & compare flights	SerpAPI
Hotel Server	Discover accommodations	SerpAPI
Event Server	Find local events & activities	SerpAPI
Geocoder Server	Location → coordinates	OpenStreetMap
Weather Server	Weather forecasts & conditions	Open-Meteo
Finance Server	Currency conversion	SerpAPI
Traffic & Crowd Server	Real-time location analysis	Computer Vision

Architecture Pattern:

- Each server is an independent MCP server
- Can be used standalone or orchestrated together
- Follows FastMCP framework standards
- Easy to add new servers

Data Storage & Conversation History

DynamoDB Role:

- Chat Sessions Table: Tracks active conversations
- Messages Table: Stores user/assistant messages
- Tool Calls Table: Audit trail of all MCP server calls

Why DynamoDB?

- Scalability: Handles concurrent conversations
- Serverless: No database management overhead
- Multi-Instance: Multiple bot instances share same data
- AWS Integration: Works seamlessly with ECS, CloudFormation

AWS Cloud Deployment Architecture

Application Layer (ECS Fargate)

- Runs a Docker container with Slack Bot + 7 MCP microservices
- Serverless containers with 1 vCPU & 2 GB RAM
- Deployed across 2 Availability Zones for high availability
- Fully managed - no EC2 servers to maintain

Data & Storage Layer

- **DynamoDB** → Stores chat sessions, messages & logs
- **S3** → File storage and automated backups
- **Secrets Manager** → Secure storage for API keys

AWS Cloud Deployment Architecture

Infrastructure Layer

- VPC with 2 public subnets (Multi-AZ)
- Security Groups & IAM for access control
- CloudWatch for monitoring & logs
- ECR as container registry

Infrastructure as Code (CloudFormation)

- CloudFormation template
- Fully version-controlled
- Environment-specific parameters
- One-click automated provisioning

Scaling & Production Monitoring

Auto-Scaling Policy

- **Scale Out:** CPU > 70% or Memory > 75% → Add tasks
- **Scale In:** CPU < 30% or Memory < 40% → Remove tasks
- **Range:** 1–5 tasks (Default: 3)

CloudWatch Monitoring

- ECS: CPU/Memory, task count, health status
- DynamoDB: Read/Write capacity, throttling, errors
- 20+ alarms for performance, scaling, and health checks

Deployment Pipeline

- Docker → ECR → CloudFormation → ECS Rolling Update
- Zero-downtime, Multi-AZ, 99.9% SLA

Key Performance Metrics

System Health

- ECS Service: ACTIVE (2/2 tasks running)
- Availability: 100% - All tasks operational
- Errors: 0 detected

Database Performance

- Read Capacity: 16 RCU (very low usage)
- Write Capacity: 349 WCU (moderate usage)
- Throttling Events: 0 - No bottlenecks

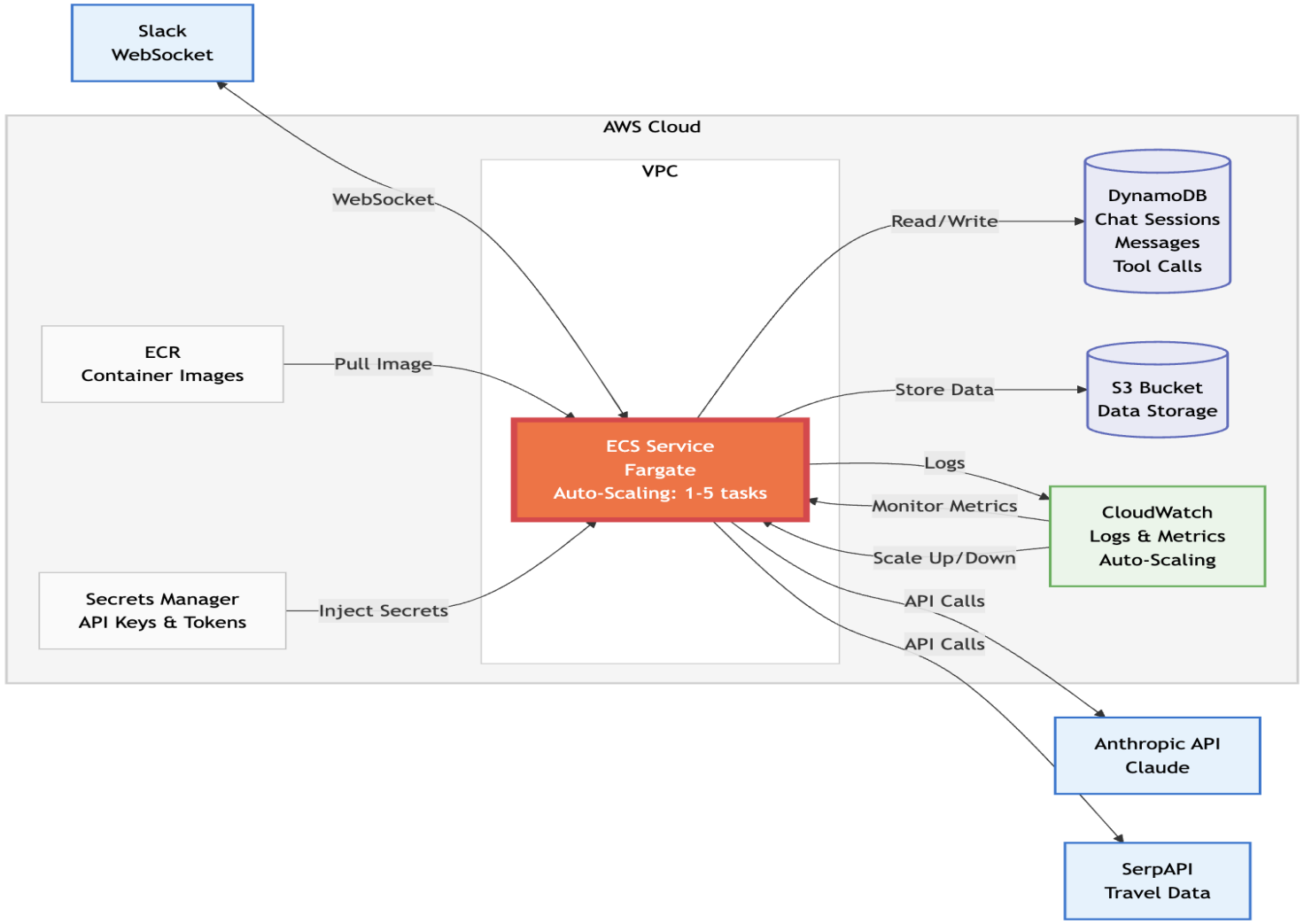
Monitoring Status

- CloudWatch Alarms: 18 configured
- Healthy Alarms: 15/18
- Overall Status: HEALTHY

Key Highlights

- Zero throttling events
- 100% availability
- Comprehensive monitoring in place

Flow Diagram - Cloud Side



Live Demo:

Live Demo:

Why TravelGenie?

FOR USERS:

- Fast: Complete itinerary in seconds
- Intelligent: Context-aware recommendations
- Budget-Conscious: Automatic currency conversion
- Weather-Aware: Optimizes activities by forecast
- Traffic-Smart: Recommends optimal visit times

FOR DEVELOPERS:

- Modular: Easy to add new servers
- Observable: Full audit trail of tool calls
- Scalable: Production-ready AWS deployment
- Extensible: MCP protocol standard
- Well-Documented: Clear architecture

FOR ORGANIZATIONS:

- Multi-Modal: Slack, Web, API access
- Secure: AWS Secrets Manager integration
- Scalable: Handles high traffic
- Cost-Effective: Pay-per-use DynamoDB

TravelGenie - The Future of Travel Planning

What We've Built:

✓Intelligent AI-powered travel planning system
✓7 specialized MCP servers
✓Multi-modal user interfaces
✓Production-ready AWS deployment
✓Context-aware conversation management

Key Innovation:

"TravelGenie orchestrates multiple services intelligently, understanding dependencies and synthesizing results into actionable travel plans."

Future Enhancements:

- Car rental integration
- Restaurant reservations
- Event ticket booking
- Mobile app
- Multi-language support

What We've Built:

✓Intelligent AI-powered travel planning system
✓7 specialized MCP servers
✓Multi-modal user interfaces
✓Production-ready AWS deployment
✓Context-aware conversation management

Key Innovation:

"TravelGenie orchestrates multiple services intelligently, understanding dependencies and synthesizing results into actionable travel plans."

Future Enhancements:

- Car rental integration
- Restaurant reservations
- Event ticket booking
- Mobile app
- Multi-language support

TECHNOLOGY STACK

CORE TECHNOLOGIES:

- Python 3.12+
- Claude AI (Anthropic)
- Model Context Protocol (MCP)
- FastMCP (MCP server framework)

EXTERNAL APIs:

- SerpAPI (Flights, hotels, events, finance)
- Open-Meteo (Weather forecasts)
- OpenStreetMap (Geocoding)
- Computer Vision (Traffic analysis)

INFRASTRUCTURE:

- AWS (ECS Fargate, DynamoDB, Secrets Manager)
- Docker
- CloudFormation
- SQLite

UI FRAMEWORKS:

- Streamlit (Web interface)
- Slack API (Bot integration)

Questions..??

Contact:

- Arunbh Yashaswi - arunbhy@umd.edu
- Abhishek Rithik Origanti - Origanti@umd.edu
- Ajaykumar Balakannan - ajayport@umd.edu
- Matheshwara Annamalai Senthilkumar - msenthi7@umd.edu

Repository: https://github.com/kautilyaa/TravelGenie

References

- Wang, S., Wang, Q., Cui, Q., & Lan, T. (2025). Artificial Intelligence in Tourism: A Systematic Literature Review and Future Research Agenda. *Sustainability*, *17*(20), 9080. https://doi.org/10.3390/su17209080

- Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., & Gonzalez, J. E. (2024). MemGPT: Towards LLMs as operating systems. arXiv. https://doi.org/10.48550/arXiv.2310.08560

- Dragoni, N., Giallorenzo, S., Lluch Lafuente, A., Mazzara, M., Montesi, F., Mustafin, R., & Safina, L. (2017). Microservices: Yesterday, today, and tomorrow. In M. Mazzara & B. Meyer (Eds.), Present and ulterior software engineering (pp. 195–216). Springer. https://doi.org/10.1007/978-3-319-67425-4_12

- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. arXiv. https://doi.org/10.48550/arXiv.2201.11903

Thank You..!